

Función de seguimiento Omnibot V.02

Universidad Veracruzana

Instituto de Investigaciones en Inteligencia Artificial



El presente documento tiene como objetivo el explicar el funcionamiento técnico de la función de seguimiento de objetos de la App Omnibot.

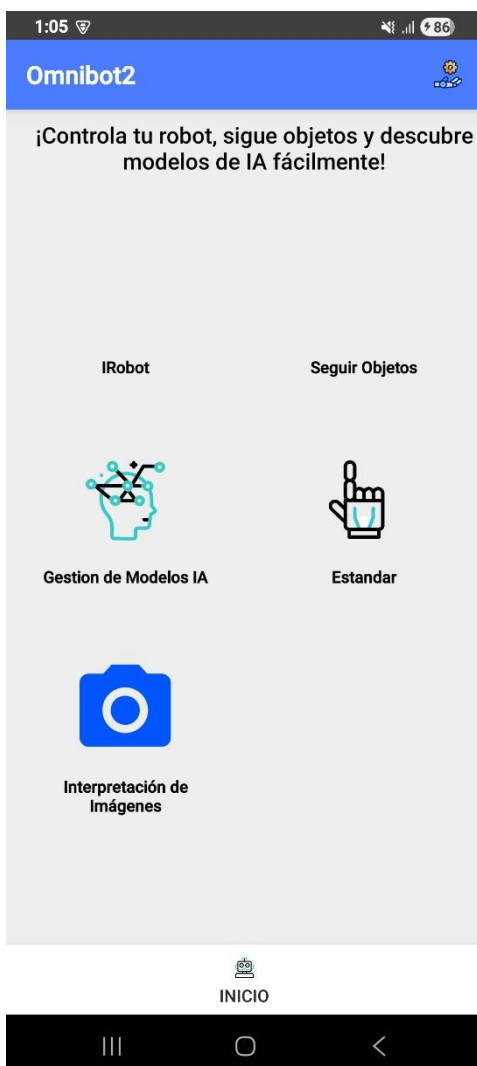
Las especificaciones técnicas para poder ejecutar por primera vez este proyecto están en el documento (Iniciar proyecto Omnibot V2.pdf).

Una vez que tengas el repositorio clonado y el proyecto actualizado compila el proyecto con tu dispositivo Android conectado mediante cable.



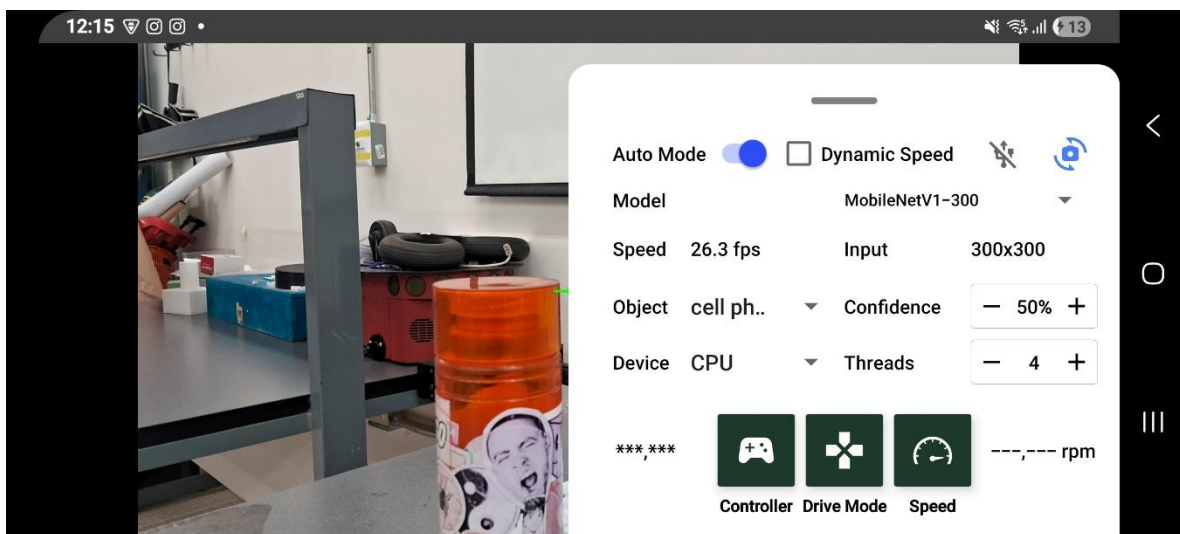
El proyecto se ejecutará en el dispositivo móvil y podrás acceder a la App Omnibot, ahí habrá un menú con todas las funciones que ofrece la App.

Abre la sección que dice “Seguir Objetos”



Al entrar a la sección de "Seguir Objetos" (Object Tracking), la aplicación nos muestra la vista de la cámara y un panel inferior deslizable con las configuraciones del sistema. Los controles más importantes para el funcionamiento del Pan-Tilt son los siguientes:

- **Interruptor "Auto Mode":** Es el control maestro. Al activarlo, el robot comenzará a mover los motores Pan-Tilt para seguir al objeto seleccionado. Si está desactivado, la cámara seguirá detectando objetos en pantalla, pero los motores no se moverán.
- **Menú Desplegable "Object":** Permite elegir qué objeto específico debe buscar y seguir la Inteligencia Artificial (ej. *cell phone*, *person*, *book*, *cup*). El robot ignorará cualquier otro objeto que no coincida con esta selección.
- **Control "Confidence" (+/-):** Define el porcentaje mínimo de seguridad que debe tener la IA para considerar que el objeto es válido. Por ejemplo, si está en 50%, la IA solo seguirá el objeto si está al menos 50% segura de que es lo que se seleccionó.
- **Conexión (Iconos USB/Bluetooth):** En la parte superior derecha del panel se muestra el estado de la conexión con el microcontrolador. Para el mecanismo Pan-Tilt por cable, el icono de **USB** debe estar en azul (conectado).
- **Configuraciones de Hardware (Device, Threads, Model):** Permiten optimizar el rendimiento del dispositivo móvil, eligiendo si el procesamiento de la IA se hará en el procesador (CPU) o tarjeta gráfica (GPU), así como el número de hilos (Threads) utilizados. Esto impacta directamente en los fotogramas por segundo (fps) del reconocimiento.



Vista de la actividad de la cámara.

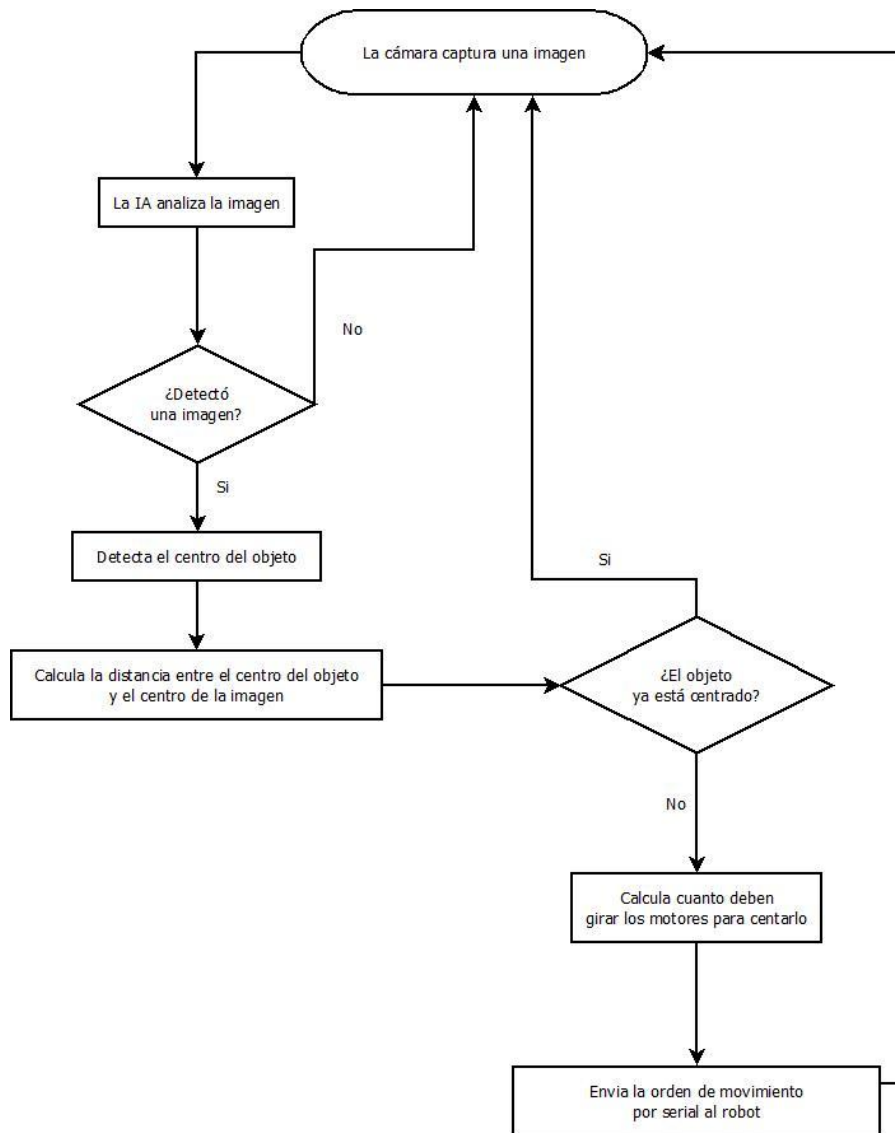


Funcionamiento en tiempo real del seguimiento de un Celular

Funcionamiento técnico y lógica de seguimiento

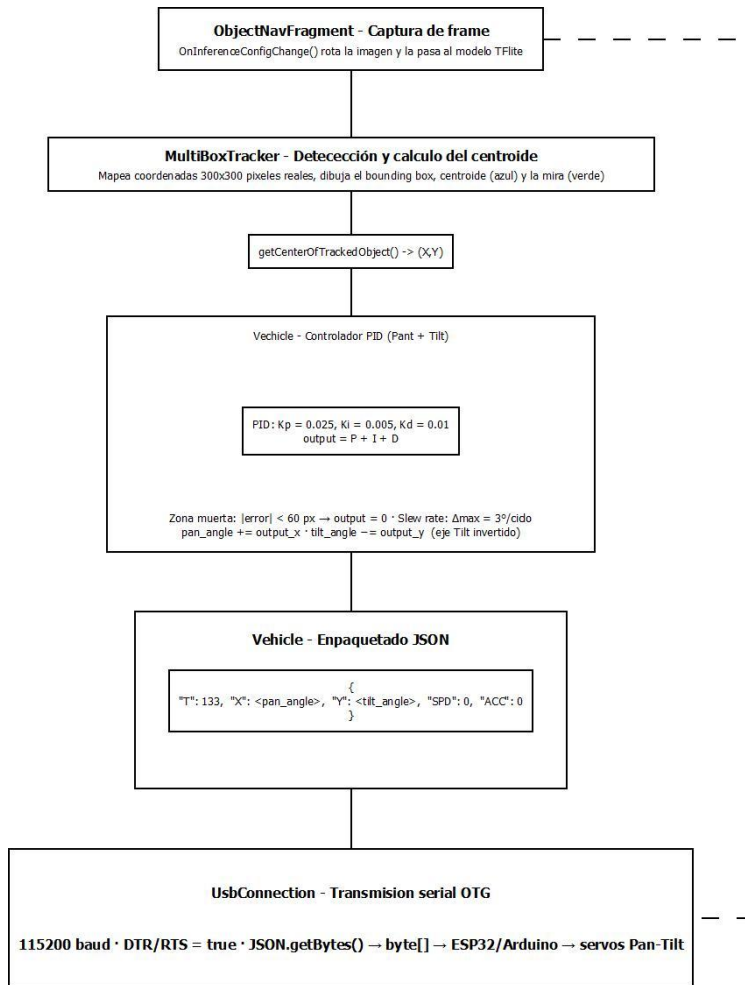
El seguimiento de objetos en la app Omnibot se logra mediante la interacción de cuatro clases principales escritas en Java, las cuales forman una tubería (pipeline) de procesamiento continuo: desde que la cámara captura un cuadro, hasta que los servomotores reciben la instrucción de moverse.

A continuación, se muestra un diagrama de flujo simple donde se ejemplifica visualmente la lógica de seguimiento en la App.



Flujo del Sistema (Pipeline)

1. **Captura y Detección:** El teléfono obtiene una imagen de la cámara y se la pasa a un modelo de IA (TensorFlow Lite).
2. **Procesamiento Visual:** Se dibuja un cuadro azul sobre el objeto detectado y se calculan las coordenadas de su centroide (punto central).
3. **Cálculo de Control (PID):** Se compara la posición del objeto con el centro de la pantalla ("la mira"). Se calcula el error matemático y un algoritmo determina cuántos grados deben moverse los motores para centrarlo.
4. **Transmisión de Datos:** Se formatea un mensaje JSON con los ángulos exactos y se envía por el cable USB a la placa controladora (ESP32/Arduino).



Clases involucradas y responsabilidades

A continuación, se detalla el papel de cada clase en este proceso:

1. ObjectNavFragment.java (El Coordinador) Es la clase maestra de la pantalla. Hereda de los componentes de cámara y coordina que la Inteligencia Artificial analice los cuadros (frames).

- **Gestión de Orientación:** Se asegura de rotar la imagen de manera dinámica para que la IA reconozca objetos sin importar si el teléfono está colocado en posición vertical u horizontal (Landscape/Portrait).
- **Anulación de ruedas:** Configura la velocidad de los motores de las llantas a cero (`vehicle.setControl(0, 0);`) para que el chasis del robot permanezca estático y el seguimiento sea exclusivo de la cabeza Pan-Tilt.
- **Puente de información:** Extrae el centro del objeto detectado y se lo envía a la clase lógica (`vehicle.receiveCenterOfTrackedObject(...)`).

2. MultiBoxTracker.java (El Procesador Visual) Se encarga de toda la capa visual de depuración que el usuario ve en la pantalla.

- **Mapeo de Coordenadas:** Transforma las medidas de la IA (que analiza una imagen comprimida de 300x300) a los pixeles reales de la pantalla del celular.
- **Renderizado:** Dibuja la caja delimitadora roja (Bounding Box), el punto central del objeto (Centroide azul) y una **Mira Verde (Crosshair)** en el centro exacto de la pantalla. Esta mira representa el *Setpoint* o punto objetivo al que el robot intenta llevar el objeto.
- **Cálculo de Centro:** A través del método `getCenterOfTrackedObject()`, expone matemáticamente la coordenada (X, Y) del objeto de mayor confianza detectado.

3. Vehicle.java (El Cerebro de Movimiento / Lógica PID) Es la representación lógica del hardware del robot. Contiene las matemáticas complejas para que el movimiento sea estable.

- **Controladores PID Independientes:** Implementa una clase interna `PIDController` (Proporcional-Integral-Derivativo) y crea dos instancias: una para el eje Pan (X) y otra para el Tilt (Y). Esto asegura que el robot responda suavemente a los cambios, frenando la aceleración para no pasarse del objetivo y evitando oscilaciones bruscas (jitter).
- **Limitador de Velocidad (Slew Rate):** Evita que los servomotores den saltos violentos al limitar el cambio máximo permitido por cada ciclo a 3 grados (`maxPanStep`).
- **Zonas Muertas (Deadzone):** Ignora pequeñas variaciones (ruido de la IA) si el objeto está a menos de 60 pixeles del centro, garantizando que el mecanismo se quede inmóvil cuando el objeto está estático.
- **Formateo de Comandos:** Calcula las posiciones finales tomando en cuenta la dirección física de los servos y empaqueta el resultado en una cadena JSON estandarizada: `{"T":133, "X":<grados>, "Y":<grados>, "SPD":0, "ACC":0}`.

4. UsbConnection.java (La Capa de Transporte) Gestiona la comunicación serial a bajo nivel mediante OTG (USB On-The-Go).

- **Configuración del Puerto:** Inicializa la velocidad de comunicación (115200 baudios) y, de forma vital para placas como ESP32, activa las señales **DTR** (Data Terminal Ready) y **RTS** (Request To Send) para mantener despierto al microcontrolador.
- **Transmisión de Bytes:** Envía la cadena JSON empaquetada por Vehicle.java convirtiéndola a un arreglo de bytes (byte[]) que viaja por el cable hacia los servomotores.

Flujo de la lógica del seguimiento

El sistema de seguimiento ejecuta un ciclo continuo de cinco etapas que se repite por cada frame capturado por la cámara.

En la primera etapa, ObjectNavFragment.java recibe el frame y aplica una corrección dinámica de orientación mediante onInferenceConfigChanged(), rotando la imagen según si el dispositivo está en modo Portrait o Landscape antes de pasarla al modelo de inteligencia artificial (TensorFlow Lite).

En la segunda etapa, MultiBoxTracker.java recibe los resultados de la inferencia. El método processResults() filtra las detecciones inválidas (sin ubicación o menores a 4px) y las almacena en la lista trackedObjects. A continuación, updateFrameToCanvasMatrix() construye la matriz de transformación que escala las coordenadas del espacio del modelo (300×300 píxeles) al espacio real de la pantalla del dispositivo. A partir del primer elemento de trackedObjects, el método getCenterOfTrackedObject() aplica esa matriz al bounding box y devuelve la coordenada (X, Y) del centroide ya mapeada a píxeles de pantalla. El método draw() se encarga adicionalmente de renderizar en cada frame el bounding box rojo con esquinas redondeadas, el centroide azul con sus coordenadas, la etiqueta del objeto con su porcentaje de confianza, y la mira verde (drawCrosshair()) en el centro exacto de la pantalla, que representa el setpoint del sistema.

En la tercera etapa, Vehicle.java recibe la coordenada (X, Y) del centroide a través de receiveCenterOfTrackedObject(), que delega inmediatamente en trackObject(). Este método calcula el setpoint como el centro geométrico del frame ($\text{frameWidth} / 2$, $\text{frameHeight} / 2$) y procesa cada eje de forma independiente. El controlador PID (PIDControlador) tiene una zona muerta interna de 10px: si el error calculado es menor a ese valor, se trata como cero para evitar que la integral acumule ruido de la IA. Superado ese umbral, se calcula la salida como la suma de los tres términos: Proporcional ($K_p = 0.025$), Integral ($K_i = 0.005$) con clamping anti-windup a ± 200 , y Derivativo ($K_d = 0.1$). La salida resultante pasa por el limitador de velocidad (slew rate), que recorta el ajuste a un máximo de ± 3 grados por ciclo. Sobre esa salida se aplica una segunda zona muerta externa

de 60px: si la distancia del centroide al centro de la pantalla no supera ese umbral, el acumulador `currentPan` o `currentTilt` no se modifica, garantizando que el mecanismo permanezca inmóvil cuando el objeto está prácticamente centrado. Para el eje Tilt, el ajuste se aplica con signo negativo (`currentTilt -= adjustmentY`) para compensar la diferencia entre el sistema de coordenadas de Android (Y crece hacia abajo) y la dirección física del servomotor (Tilt positivo apunta hacia arriba). Finalmente, los acumuladores son limitados mecánicamente: `currentPan` a $[-180^\circ, +180^\circ]$ y `currentTilt` a $[-30^\circ, +90^\circ]$.

En la cuarta etapa, `mandarPanTilt()` aplica un control de flujo antes de transmitir: si han transcurrido menos de 40 ms desde el último envío (equivalente a 25 Hz), el método retorna sin hacer nada para no saturar el buffer serial del microcontrolador. Si el intervalo se cumple, construye la cadena JSON: `{"T": 133, "X": <currentPan>, "Y": <currentTilt>, "SPD": 0, "ACC": 0}`. Los campos SPD y ACC se envían siempre en cero porque el control de velocidad y aceleración es gestionado por el propio firmware del ESP32.

En la quinta y última etapa, `mandarBytesAlDispositivo()` verifica el tipo de conexión activo y, si es USB, delega en `UsbConnection.send()`. Este método convierte la cadena JSON a un arreglo de bytes (`getBytes(UTF-8)`) y lo escribe en el puerto serial OTG a 115200 baudios. Las señales DTR y RTS se mantienen en true desde la apertura de la conexión para garantizar que el ESP32 permanezca activo. Al llegar al microcontrolador, el campo T:133 identifica el tipo de comando como control Pan-Tilt absoluto, y los campos X e Y son usados directamente para posicionar los servomotores.

Explicación de las funciones de `ObjectNavFragment.java`

Esta clase actúa como coordinador general de la pantalla de seguimiento. Hereda de los componentes de cámara del framework OpenBot y es responsable de iniciar y mantener el ciclo de inferencia. Su función más relevante para el seguimiento es la gestión dinámica de la orientación: al detectar un cambio en la posición del sensor, recalcula el ángulo de rotación necesario y lo aplica a cada frame antes de pasarlo al modelo, asegurando que la IA reciba siempre la imagen correctamente orientada sin importar si el teléfono está en modo Portrait o Landscape.

Otra responsabilidad clave es suprimir el control de las llantas del chasis (`vehicle.setControl(0, 0)`), de modo que durante el seguimiento el robot permanece físicamente estático y todo el movimiento recae exclusivamente en el mecanismo Pan-Tilt. Por último, actúa como puente de información: extrae el centroide del objeto detectado desde `MultiBoxTracker` y lo entrega a `Vehicle` a través de `vehicle.receiveCenterOfTrackedObject()`, conectando la capa visual con la capa de control.

Explicación de las funciones de MultiBoxTracker.java

Esta clase gestiona toda la capa visual y el cálculo de posición del objeto. Recibe los resultados crudos del modelo TFLite a través de `trackResults()`, que internamente llama a `processResults()`: este método filtra detecciones inválidas (sin rectángulo de ubicación o menores a 4px), mapea cada resultado al espacio de pantalla usando la matriz calculada por `updateFrameToCanvasMatrix()`, y almacena los objetos válidos en la lista `trackedObjects`.

La función `draw()` se ejecuta en cada frame y dibuja tres elementos sobre el canvas: el bounding box rojo con esquinas redondeadas alrededor del objeto detectado, el centroide azul en el centro geométrico de esa caja junto con sus coordenadas en píxeles, y la mira verde central mediante `drawCrosshair()`, que traza dos líneas perpendiculares y un círculo en el punto exacto (`canvas.getWidth()/2`, `canvas.getHeight()/2`). Esta mira representa visualmente el setpoint del sistema de control.

La función `getCenterOfTrackedObject()` es la interfaz de salida más importante de la clase: toma el primer elemento de `trackedObjects` (el de mayor prioridad), le aplica la transformación `frameToCanvasMatrix` para obtener coordenadas en píxeles de pantalla real, y devuelve un objeto `Point(X, Y)` que `Vehicle.java` usará como entrada del PID. Si no hay ningún objeto detectado, retorna null y el robot no ejecuta ningún movimiento.

Explicación de las funciones de Vehicle.java

Esta clase representa la entidad lógica del robot y concentra toda la matemática del sistema de control Pan-Tilt. Contiene una clase interna `PIDControlador` que implementa el algoritmo Proporcional-Integral-Derivativo con dos mecanismos de protección: una zona muerta interna de 10px que evita acumular la integral con variaciones mínimas del sensor, y un clamping anti-windup que limita el valor de la integral a ± 200 para prevenir que se desborde en situaciones donde el robot no puede alcanzar el objetivo.

La función principal del seguimiento es `trackObject()`. Calcula el error de posición en ambos ejes, aplica un PID independiente para cada uno, limita la salida al slew rate de ± 3 grados por ciclo, y solo actualiza los acumuladores `currentPan` y `currentTilt` si el error supera los 60px de zona muerta externa. El eje Tilt aplica el ajuste con signo negativo para compensar la inversión entre el sistema de coordenadas de la pantalla (Y hacia abajo) y la dirección física del servomotor. Antes de enviar, ambos acumuladores son recortados a sus límites mecánicos seguros: Pan en $[-180^\circ, +180^\circ]$ y Tilt en $[-30^\circ, +90^\circ]$.

La función `mandarPanTilt()` implementa un throttling de 40ms que evita saturar el buffer serial del microcontrolador. Solo cuando ese intervalo se cumple, construye y envía el JSON. La función `reiniciaPanTilt()` resetea los acumuladores a cero, limpia la memoria integral de ambos PIDs y envía el comando de posición cero al hardware, útil al reconectar

o cambiar de modo. Finalmente, `mandarBytesAlDispositivo()` abstrae el tipo de transporte activo (USB o Bluetooth) y delega la transmisión al módulo correspondiente.

Explicación de las funciones de `UsbConnection.java`

Esta clase gestiona la comunicación serial de bajo nivel entre el dispositivo Android y el microcontrolador (ESP32/Arduino) mediante el protocolo OTG (USB On-The-Go). Utiliza la biblioteca `felhr/usbserial` para abstraer el acceso al hardware serial.

La función `startUsbConnection()` inicia el proceso registrando dos receptores de eventos del sistema operativo: uno para cuando el usuario otorga o deniega el permiso USB, y otro para cuando el cable es desconectado físicamente. Luego enumera los dispositivos conectados al puerto OTG y, si ya existe permiso, llama directamente a `startSerialConnection()`; de lo contrario, solicita permiso al usuario mediante un `PendingIntent`.

La función `startSerialConnection()` abre el dispositivo y configura todos los parámetros del puerto: 115200 baudios, 8 bits de datos, 1 bit de parada, sin paridad y sin control de flujo por software. La configuración más crítica es la activación de las señales DTR (Data Terminal Ready) y RTS (Request To Send) en `true`, necesaria para que ciertos chips como el CP2102 del ESP32 inicien correctamente la transmisión de datos. Una vez configurado el puerto, registra el callback de lectura, que acumula los bytes recibidos en un buffer de texto y procesa mensajes completos cada vez que detecta el carácter `\n`, distribuyéndolos al resto de la aplicación mediante `LocalBroadcastManager`.

La función `send()` es el punto de salida de datos: verifica que la conexión esté abierta y no esté ocupada, marca la bandera `busy` para evitar escrituras simultáneas, llama a `serialDevice.write(byte[])` para transmitir los bytes al microcontrolador, y libera la bandera al terminar. La función `stopUsbConnection()` cierra el puerto serial, libera la conexión y desregistra todos los receptores de eventos para evitar fugas de memoria.